

Arquitetura de Mensageria

Importa Aí — Sistema de Gestão e Rastreamento de Encomendas Internacionais

PROJETO

Importa Aí

DISCIPLINA

Mensageria e Streams

AUTOR

Higor Paulo Costa

BROKER

RabbitMQ 3.13

DATA

Abril / 2026

CURSO

ADS — 4º Período — PUC Goiás

Visão Geral

O Importa Aí utiliza **RabbitMQ** como broker de mensagens para desacoplar as operações de escrita do frontend da persistência no banco de dados. Toda operação de escrita segue o padrão **fire-and-forget**: o backend publica um evento na fila e retorna HTTP 202 imediatamente, sem aguardar a persistência — garantindo responsividade da interface (RN03 da ERS).

Princípio central: o frontend jamais aguarda confirmação de escrita no banco. O backend retorna HTTP 202 após publicar o evento. A persistência ocorre de forma assíncrona via consumer.

Topologia RabbitMQ

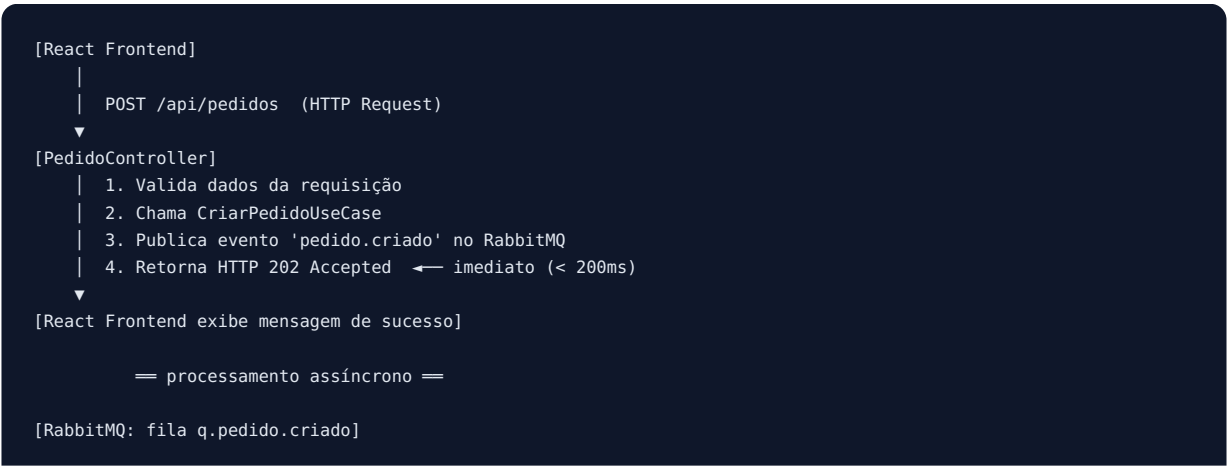
Exchange Principal

NOME	TIPO	DURABLE	DESCRIÇÃO
importaai.events	direct	true	Exchange principal para todos os eventos de domínio
importaai.events.dlq	direct	true	Exchange de Dead Letter Queue para mensagens com falha

Filas e Bindings

ROUTING KEY	FILA CONSUMIDORA	DLQ	CONSUMER
pedido.criado	q.pedido.criado	q.pedido.criado.dlq	PedidoCriadoConsumer
pedido.atualizado	q.pedido.atualizado	q.pedido.atualizado.dlq	PedidoAtualizadoConsumer
rastreamento.atualizado	q.rastreamento.atualizado	q.rastreamento.atualizado.dlq	RastreamentoConsumer
notificacao.usuario	q.notificacao.usuario	q.notificacao.usuario.dlq	NotificacaoConsumer

Fluxo Principal: Criar Pedido



```
|
▼
[PedidoCriadoConsumer]
| 1. Verifica idempotência (tabela evento_processado)
| 2. Persiste pedido no MySQL com status PROCESSANDO
| 3. Publica evento 'notificacao.usuario'
▼
[NotificacaoConsumer]
| 1. Identifica o usuário dono do pedido
| 2. Envia mensagem via WebSocket (STOMP)
| 3. Persiste notificação no histórico
▼
[React Frontend – Toast de confirmação em tempo real]
```

Idempotência de Eventos

Para garantir que o reprocessamento de uma mensagem não gere efeitos colaterais (RN04 da ERS), cada consumer verifica uma tabela de controle antes de processar qualquer mensagem.

Tabela `evento_processado`

COLUMNA	TIPO	DESCRIÇÃO
<code>id</code>	UUID	Identificador interno
<code>exchange</code>	VARCHAR	Nome do exchange RabbitMQ
<code>routing_key</code>	VARCHAR	Routing key da mensagem
<code>message_id</code>	VARCHAR	Header MessageId da mensagem RabbitMQ
<code>processado_em</code>	TIMESTAMP	Momento do processamento

A constraint `UNIQUE(exchange, routing_key, message_id)` garante que uma mesma mensagem nunca seja processada duas vezes. Em caso de reprocessamento, o consumer descarta silenciosamente.

Dead Letter Queue (DLQ)

Cada fila principal possui uma DLQ correspondente configurada via argumento `x-dead-letter-exchange`. Uma mensagem é encaminhada para a DLQ quando o consumer lança uma exceção após 3 tentativas de reentrega. Isso evita que mensagens problemáticas bloqueiem o processamento das demais.

Configuração de retry

PARÂMETRO	VALOR
Máximo de tentativas	3
Intervalo inicial entre tentativas	1 segundo
Multiplicador de backoff	2x (1s → 2s → 4s)

PARÂMETRO	VALOR
Destino após esgotamento	DLQ da fila correspondente

Integração com WebSocket

O `NotificacaoConsumer` atua como ponte entre o RabbitMQ e os clientes conectados via WebSocket (STOMP/SockJS). Após persistir a notificação, ele envia a mensagem para o canal privado do usuário: `/user/{userId}/queue/notificacoes`. O frontend React é subscrito a esse canal no momento do login e exibe toasts em tempo real sem necessidade de polling ou refresh.